

Python Data Science Project

Frontier AI Models: Productivity and Cost Analysis



Vedant Daga

Grade 9

06/11/2026

Section 1: Project Introduction

Project Overview

For this EECS Data Science project, I wanted to use my data analysis skills and graphing skills to a topic that I enjoy learning about. Artificial Intelligence models are becoming more and more popular in fields like education, programming, and everyday life. I decided to compare different flagship AI models from different AI companies and see how they compare to each other.

When I started this project, I only tested how different text models compared to each other, but I soon realized that AI models can do much more than just that. There are image generation models and video generations models too.

This report is broken up into three main sections, one for text, one for image, and one for video. Each section has its own set of models, and its own group of different benchmarks so it's a fair comparison to find the best model in each category.

Research Question

- How do frontier AI models differ in performance across research, analytical reasoning, and coding benchmarks?
- Do certain AI models have specific strengths in certain tasks instead of just being good at everything?
- How do the costs differ between models, both training costs and token costs?
- For an image and video model, how does the quality compare to each other, and how much does the quality compare to the price?

The Models

I'm going to be testing these models in these three different groups.

Text Generation Models

- OpenAI's GPT-5.5
- Anthropic's Claude Opus 4.8
- Anthropic's Claude Sonnet 4.6
- Anthropic's Claude Fable 5
- Google's Gemini 3.1 Pro
- Google's Gemini 2.5 Pro
- xAI's Grok 4.3

Image Generation Models

- OpenAI's GPT Image 1.5
- OpenAI's GPT Image 2
- Google's Nano Banana 2
- Google's Nano Banana Pro
- Google's Imagen 4 Ultra
- Google's Imagen 4 Fast
- Black Forest Labs' Flux 2 Pro
- ByteDance's Seedream 4.0
- xAI's Grok Imagine
- Ideogram's Ideogram 3.0
- Recraft's Recraft V4

- HiDream's HiDream-O1-Image-1.5
- Midjourney's Midjourney v7

Video Generation Models

- Google's Veo 3.1
- OpenAI's Sora 2
- OpenAI's Sora 2 Pro
- Kuaishou's Kling 3.0
- ByteDance's Seedance 2.0
- Alibaba's Happy Horse 1.0
- Runway's Gen-4.5
- xAI's Grok Imagine
- Lightricks' LTX-2 Pro
- Pika's Pika 2.5

The Goal

The main goal of this project was to understand how different AI models compare to each other in metrics like performance, costs, and other factors. More specifically, I want to figure out which AI models are the best in certain categories and how they are better suited for certain tasks. I also wanted to look at how much each model costs, both token costs and training costs. I also wanted to compare how different image and video generating models compare to each other. I also

want to create a visual way to understand all of this data, and understand how each model differs from each other.

The Dataset

Instead of finding an online dataset, I decided to create my own custom dataset. I researched each model's benchmark scores for different categories and their pricing by going through each company's release logs and their pricing pages, and I also used trustworthy AI rankings sites like Artificial Analysis to compare everything fairly. Since I had three different tropes of models, I ended up making three different types of datasets, one for text models, one for image models, and one for video models.

For the text models, the main thing I used was the Artificial Analysis INtelligence Index. This metric was really important because it runs the exact same set of benchmarks on every model, so every model can be fairly compared together. Instead of mixing different benchmark tests together. I also tracked individual benchmarks (MMLU, GPQA Diamond, and SWE-bench Verified), token costs, training costs, and metrics like output speed.

For image and video models, the main benchmark I used was called Arena Elo. This score comes from people voting on which output they think looks better in blind comparisons, kind of like the ranking system in chess. I used Arena Elo because there isn't really one single test for image or video quality the way there is text, so human preference is the most consistent and trustworthy way to compare them. I also tracked specs like resolution, clip length, and cost.

After gathering all the data, I entered everything into Pandas DataFrames and saved each one as its own .csv file.

Libraries and Tools

1. **Python** - the programming language that I used for the whole project, both building the datasets and for creating the graphs.
2. **Google Colab** - the platform where I ran all of my code and made the graphs
3. **Pandas** - used to build the datasets with DataFrames, organize everything into a clean .csv file, and to pull specific values I need for each graph
4. **Plotly** - used for most of my graphs, like bar charts, line graphs, and scatter plots. It made it easy to label the axes and compare each model side by side, and the charts are interactive
5. **Seabord** - used for the heatmaps, since Seaborn is really good with color based charts
6. **Matplotlib** - used for radar chart because that shape works better in Matplotlib
7. **NumPy** - used for creating evenly spaced angles for the radar charts

I decided to use three different graphing libraries instead of just one because each library has its own strengths and is better at different types of charts, and using three of them gave my project more variety.

How Does it Work

1. Step One: Importing the Libraries and Setting the Colors

- a. The first code block importing all the libraries I needed, and setting each company to a specific color. I did this because it's easier to compare models between companies and company versus company

2. Step Two: Building the Dataset

- a. Next, I entered all the benchmark scores and the pricing information into Python using a dictionary, and then turned each one into a proper .csv file using Pandas. I did this three times, one for each classification of models.

3. Step Three: Creating the Visualizations

- a. Finally I loaded each dataset and used Plotly, Seaboard, and Matplotlib to make each graph for all the metrics. Most of the charts have the models on the x-axis and the score or cost on the y-axis. I also added additional styling to make them cleaner and easier to use.

Section 2: Text Generation Models

This section compares 7 different flagship text generation models, and compares how smart they are, how well they perform on benchmarks, their overall cost, and how fast they respond

The Text Dataset

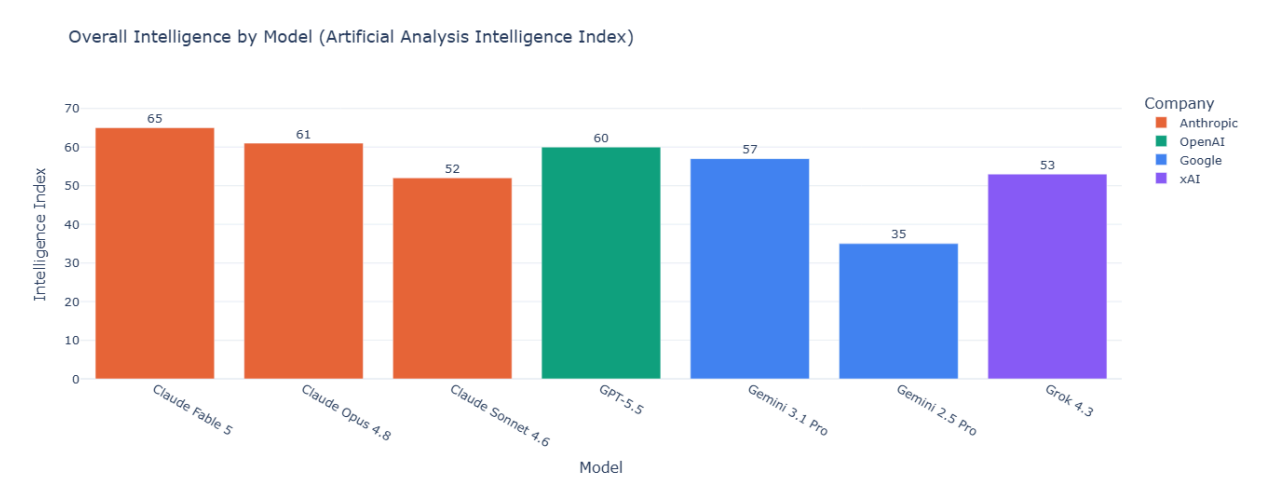
Benchmark Scores

Model	Company	AA Intelligence Index	Coding (SWE-bench)	Data Analysis (GPQA)	Research (MMLU)
Claude Fable 5	Anthropic	65	95.0	95.0	91.5
Claude Opus 4.8	Anthropic	61	88.6	93.6	92.0
GPT-5.5	OpenAI	60	88.7	93.2	92.4
Gemini 3.1 Pro	Google	57	80.6	94.3	90.1
Grok 4.3	xAI	53	70.8	90.1	91.0
Claude Sonnet 4.6	Anthropic	52	79.6	74.1	91.8
Gemini 2.5 Pro	Google	35	63.8	84.0	89.8

Cost and Specs

Model	Training Cost (\$M)	Input (\$/1M tokens)	Output (\$/1M tokens)	Speed (tokens/sec)	Knowledge Cutoff
Claude Fable 5	750	10.00	50.00	60.3	January 2026
Claude Opus 4.8	500	5.00	25.00	57.5	January 2026
GPT-5.5	500	5.00	30.00	53.2	December 2025
Gemini 3.1 Pro	350	2.00	12.00	110.1	January 2025
Grok 4.3	500	1.25	2.50	143.5	December 2025
Claude Sonnet 4.6	200	3.00	15.00	44.5	August 2025
Gemini 2.5 Pro	250	1.25	10.00	127.7	January 2025

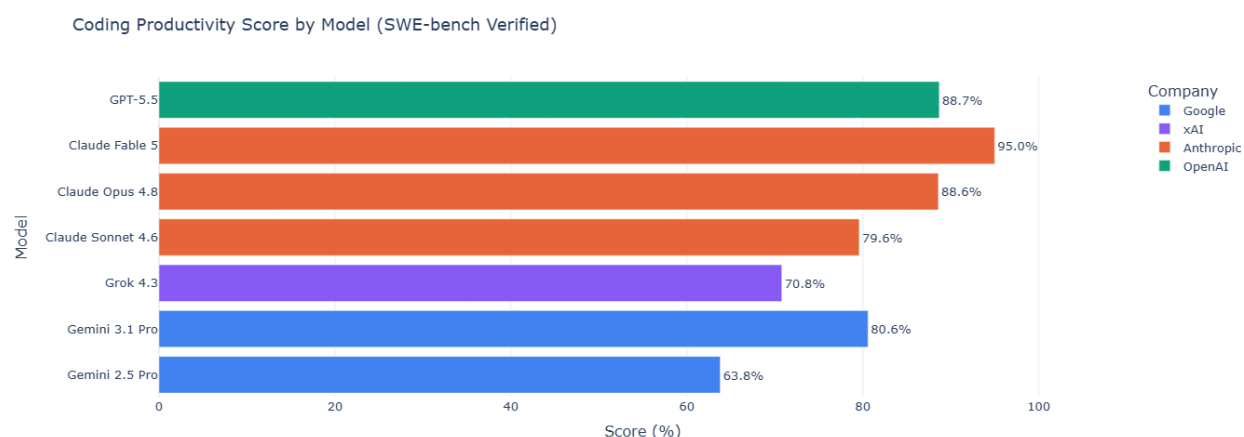
Graph 1: Overall Intelligence (Artificial Analysis Intelligence Index)



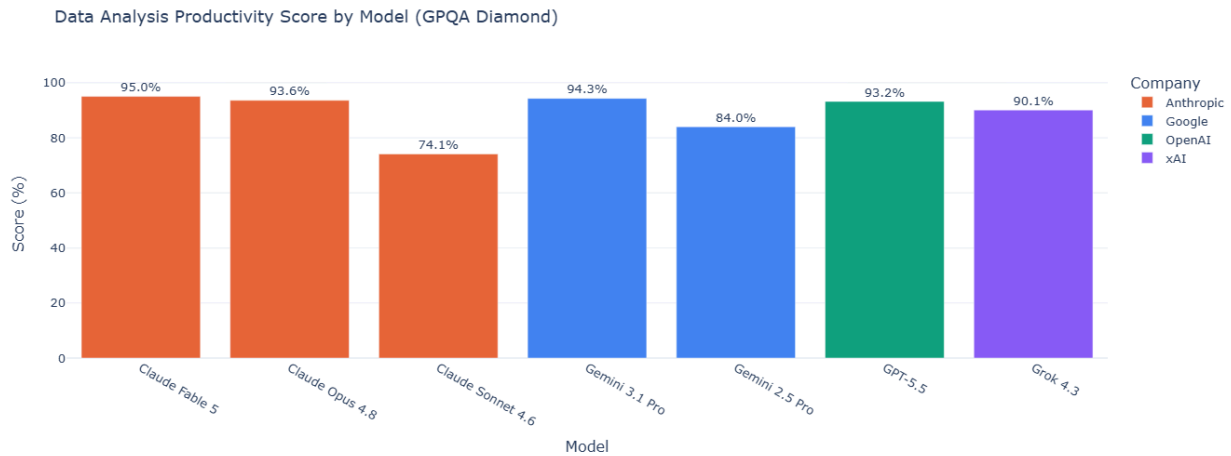
This graph shows the overall intelligence for each model, which is one of the most important benchmarks to 61, fairly comparing each model and to make sure they are tested in the same way. Claude Fable 5 came out on top with a 65, followed by Claude Opus 4.8 at GPT 5.5 at 60.

Gemini 2.5 Pro was the lowest score at 35, which does make sense as it is the oldest model in the testing group. One thing I noticed is that the top three sports are really close together, meaning that the best models aren't that far apart.

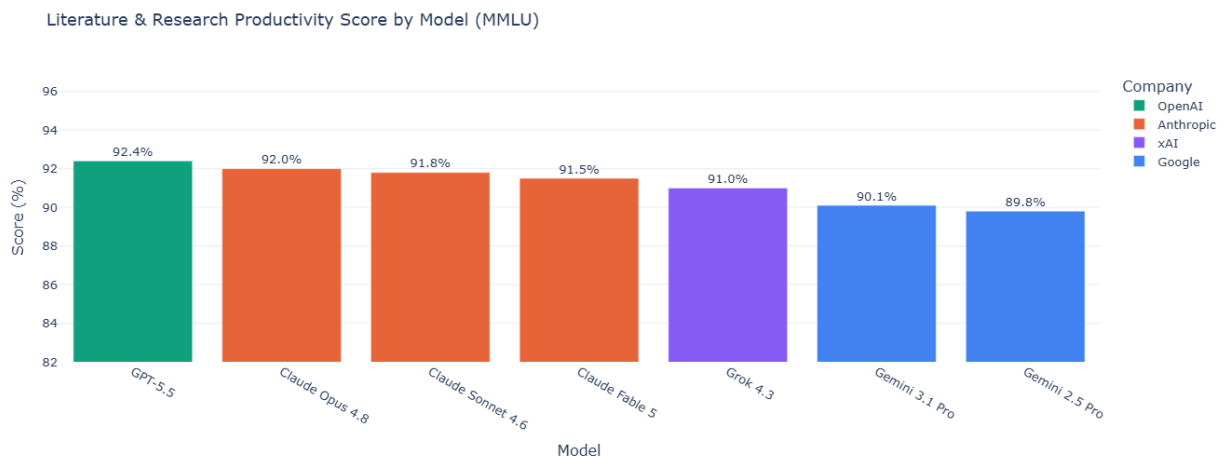
Graph 2: Coding Productivity (SWE-bench Verified)



This graph shows how well each model was able to code on real-world programming tasks. This graph has one of the highest variability rates compared to the other benchmarks going from 63.8% to 95%, because coding depends heavily on the model's exact abilities, not the general reasoning. Claude Fable 5 was clearly the best model in this category with 95%, and GPT 5.5 and Claude Opus 4.8 were right behind it. Again, Gemini 2.5 Pro was the lowest showing that newer models are getting progressively better at coding tasks

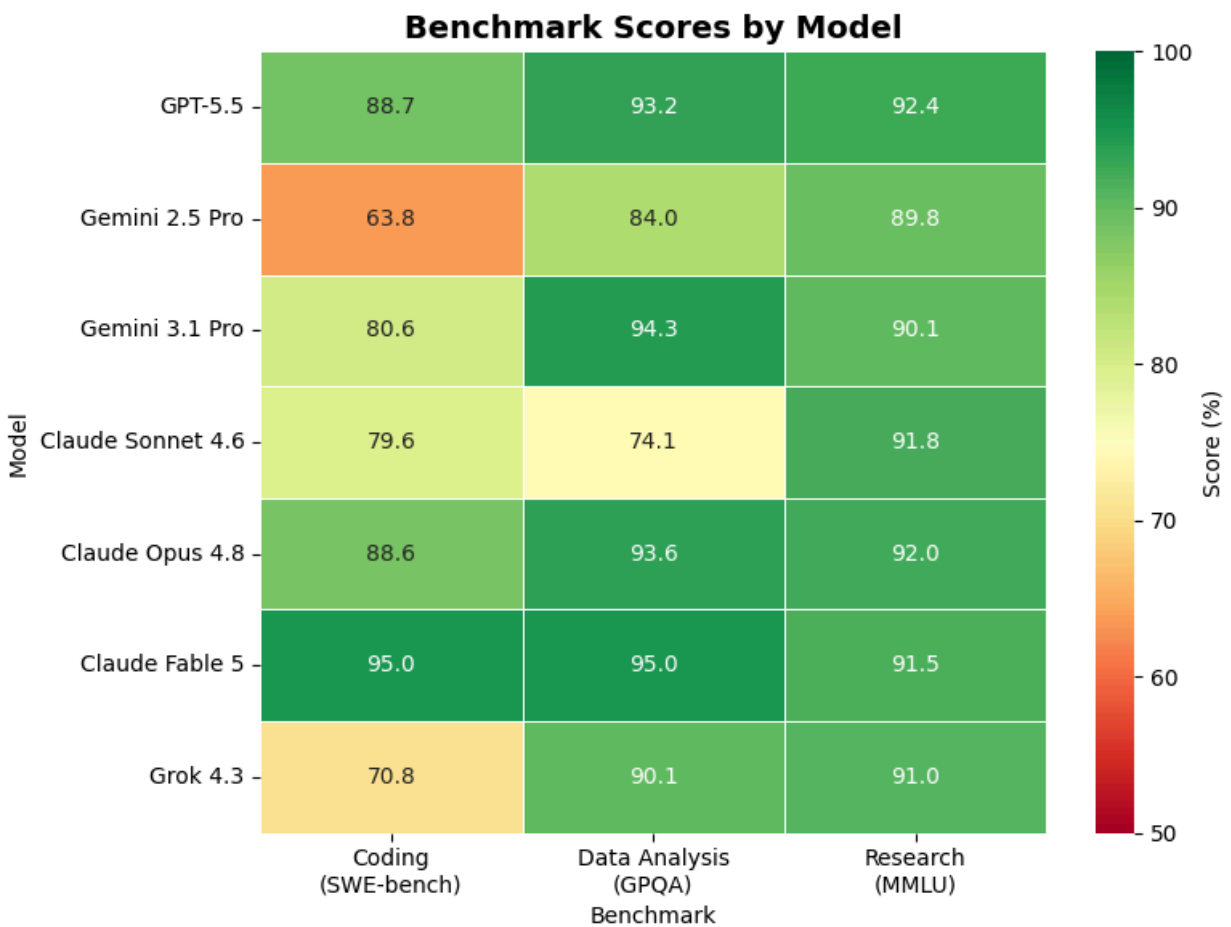
Graph 3: Data Analysis Productivity (GPQA Diamond)

This graph shows how well each model was able to reason on advanced scientific and analytical questions. The top scores were pretty similar, with Claude Fable 5, Gemini 3.1 Pro, Claude Opus 4.8, and GPT 5.5 all bunched up between 93% and 95%. The two models that performed the worst were Gemini 2.5 Pro which was expected as its the oldest model in this dataset, but also Claude Sonnet 4.6 was also extremely low at 74.1%, which is interesting because Sonnet did fine in the other categories.

Graph 4: Literature and Research Productivity (MMLU)

This graph shows how well each model completed the MMLU benchmark, which measures general knowledge and reasoning. The scores here were all pretty similar, basically all between 89.8% to 92.4%, because most models perform about the same on general knowledge tasks. GPT 5.5 was the highest score, but the gap between the scores in this test is so small, which shows how models are maxing out these tests more and more often.

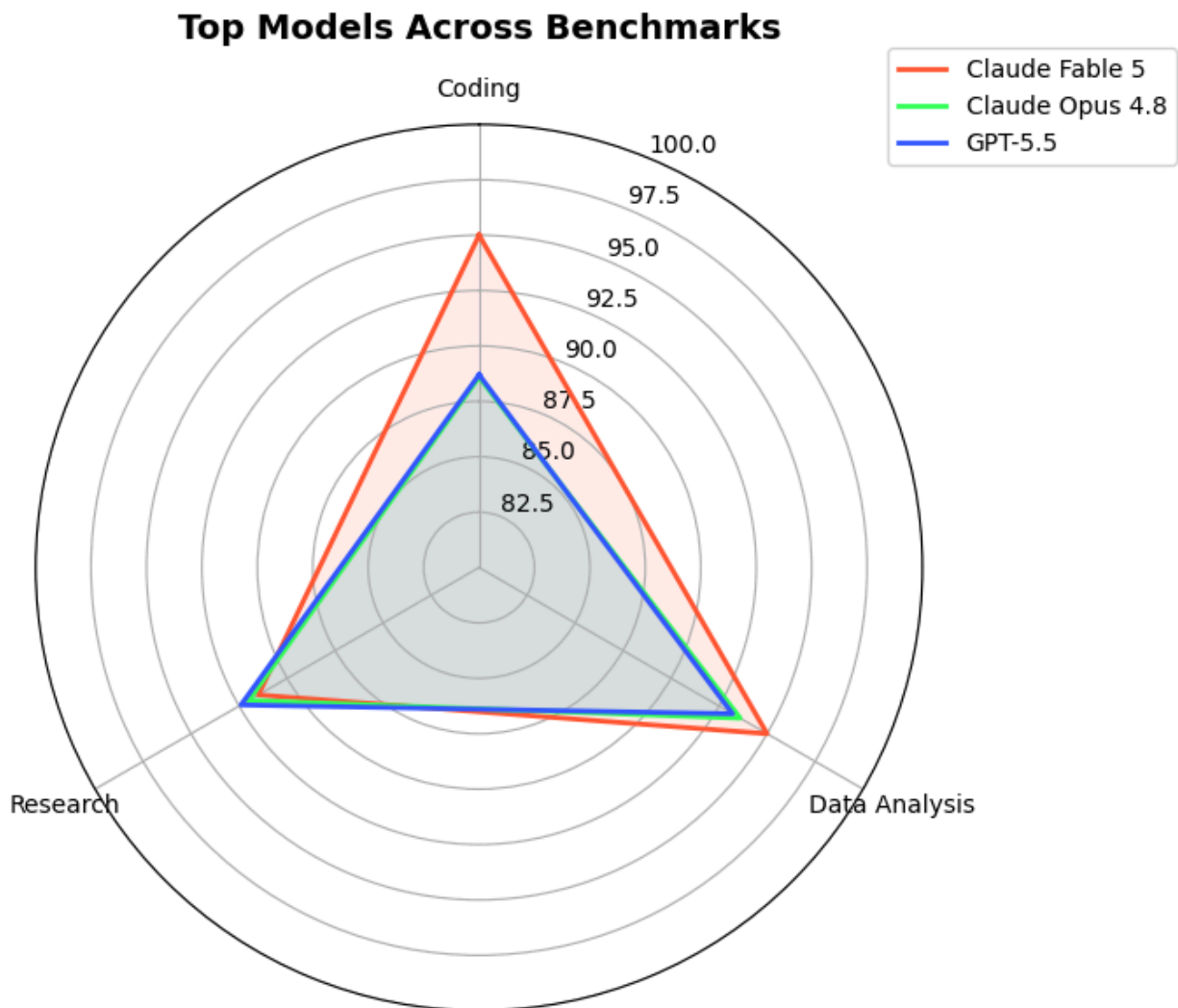
Graph 5: Benchmark Heatmap



This heatmap puts all three benchmarks scores into one chart so it's easy to spot patterns at a glance. The colors go from red for the lower scores to green to the higher scores, so a model that

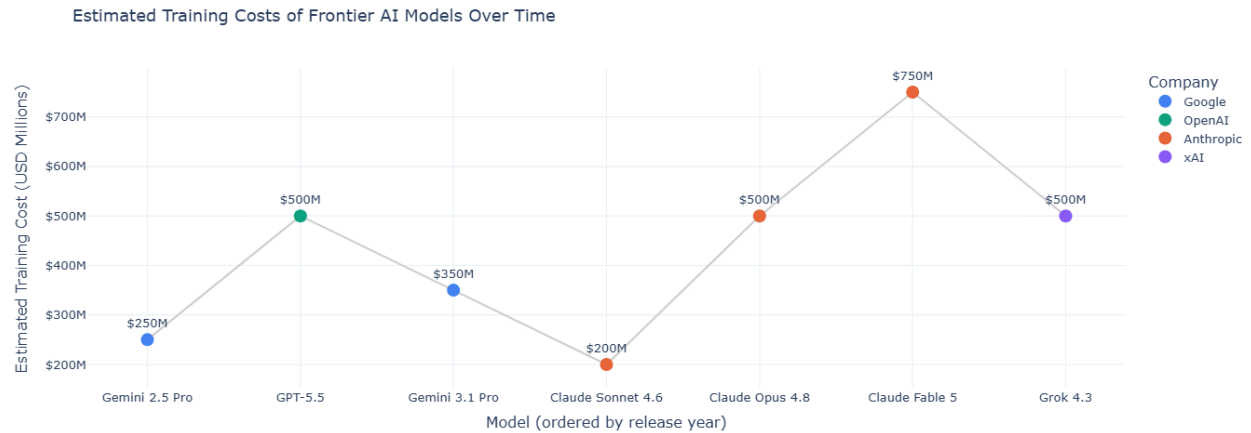
is green across the entire row is strong everywhere. Claude Fable 5 is the greenest overall, while Gemini 2.5 Pro was the most red, which lines up with everything from the other graphs.

Graph 6: Top Models Radar



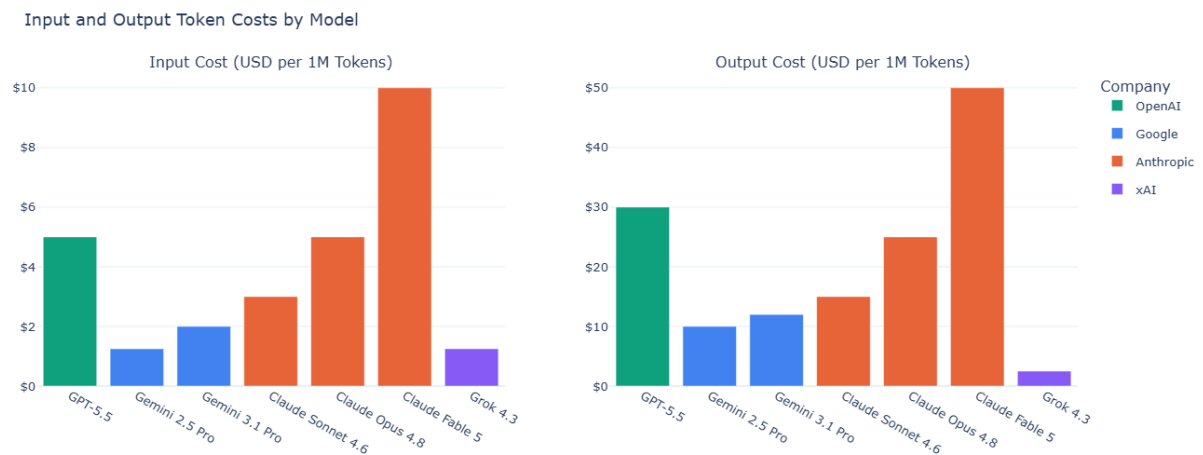
This radar chart compares the three best models by intelligence (Claude Fable 5, Claude Opus 4.8, and GPT 5.5) across three benchmarks at the same time. The shapes show which model is strong, and each of the models have a similar shape, but Claude Fable 5's shape is slightly bigger, especially on the coding side, which is its strongest aspect.

Graph 7: Overall Training Costs



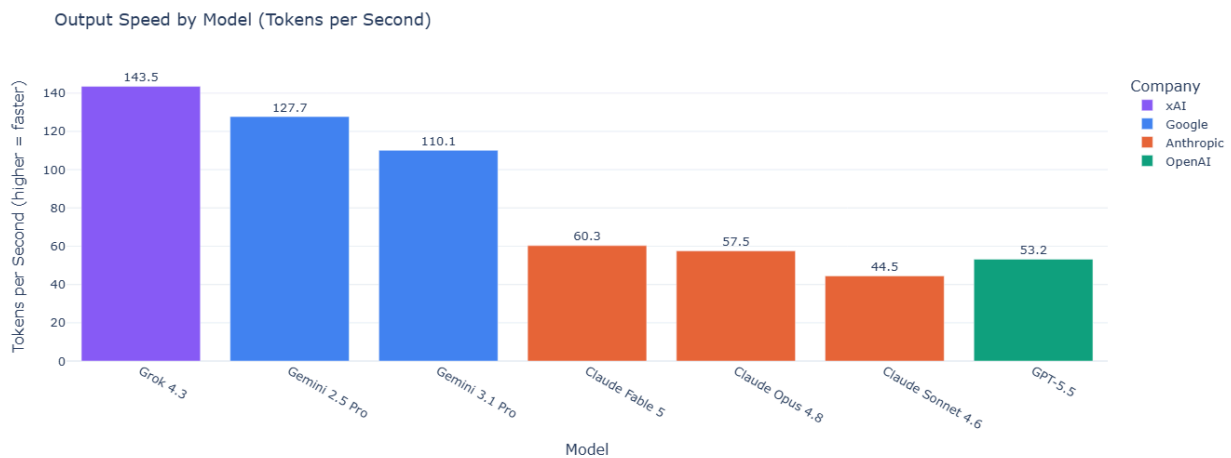
This graph shows the estimated costs to train each model. Training costs are a combination of computing power, hardware like GPUs, and the time it takes to actually build the model. A clear trend is that the more powerful models usually cost more to train, with Claude Fable 5 being the most expensive at \$750M, and the hardware costs skyrocket because they need more and more H100, H200, and Blackwell GPUs.

Graph 8: Token Costs



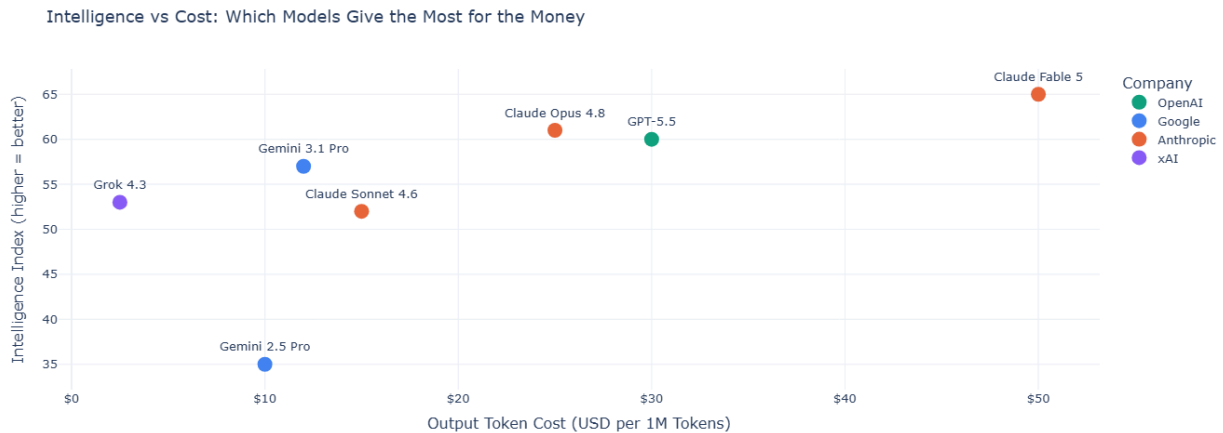
This graph shows how each model costs to use. The left side is the input cost, which is how much tokens you type or how much each prompt costs. The right side is output costs, which shows how much each token the model writes back costs. Output is almost always more expensive than input. Both sides are measured in USD per 1 million tokens. A million tokens is around 750,000 words or around 9 full books. Claude Fable 5 was the most expensive model costing \$10 input and \$50 output, while Grok 4.3 was super cheap with only \$2.50 for output costs.

Graph 9: Output Speed



This graph shows how fast each model writes in response, measured in tokens per second. Something I found really interesting is that the most capable models were some of the slowest. Grok 4.3 was the fastest at 143.5 tokens per second and Gemini 2.5 Pro was close behind, while top intelligent models like Fable 5 and Opus 4.8 were a lot slower. This shows the tradeoff between how smart a model is and how fast it answers.

Graph 10: Intelligence vs Cost



This scatter plot compares each model's intelligence against its output costs, so I can see which models give the most value for the money. The models in the top left are the best deals because they are smart and cheap, while the ones in the bottom right are expensive without being the smartest. Grok 4.3 was the best value model because it was a solid intelligence score, and the cheapest output cost. Claude Fable 5 is the smartest model but also the most expensive, so it's powerful but not the best value.

What I Found

For the text models, there is no best model. Claude Fable 5 was the most intelligent and best at codeine, but it was also the most prescient to use. Grok 4.3 was not the sparkest, but it was easily the best value because it is so cheap and still really fast. Also, the newer models were generally better than older ones, which is why Gemini 2.5 Pro fell behind. It really comes down to what you actually need the model for.

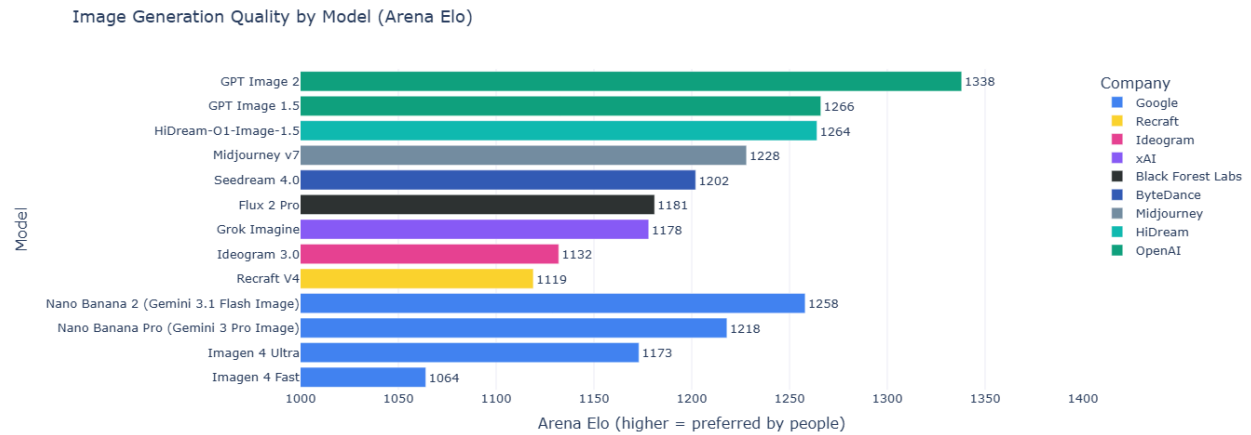
Section 3: Image Generation Models

This section compares top image generation models. Since there isn't one single test for image quality, the main benchmark is Arena Elo, which comes from people voting on which generated image looks better.

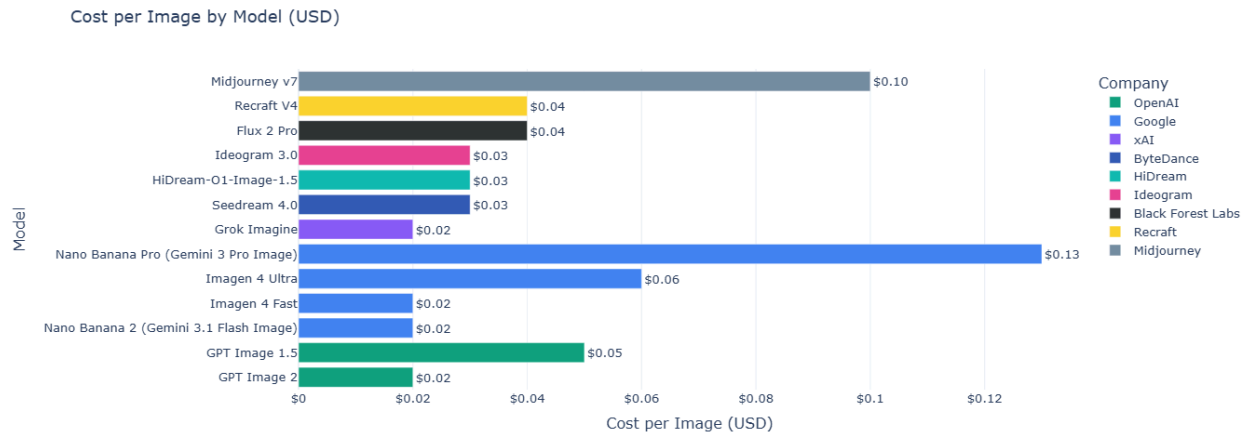
The Image Dataset

Model	Company	Arena Elo	Cost per Image	Max Resolution	Weights
GPT Image 2	OpenAI	1338	\$0.02	4K	Proprietary
GPT Image 1.5	OpenAI	1266	\$0.05	4K	Proprietary
HiDream-O1-Image-1.5	HiDream	1264	\$0.03	2K	Open
Nano Banana 2	Google	1258	\$0.02	2K	Proprietary
Midjourney v7	Midjourney	1228*	\$0.10	2K	Proprietary
Nano Banana Pro	Google	1218	\$0.13	4K	Proprietary
Seedream 4.0	ByteDance	1202	\$0.03	4K	Proprietary
Flux 2 Pro	Black Forest Labs	1181	\$0.04	4K	Proprietary
Grok Imagine	xAI	1178	\$0.02	2K	Proprietary
Imagen 4 Ultra	Google	1173	\$0.06	2K	Proprietary
Ideogram 3.0	Ideogram	1132*	\$0.03	2K	Proprietary
Recraft V4	Recraft	1119	\$0.04	4K	Proprietary
Imagen 4 Fast	Google	1064	\$0.02	2K	Proprietary

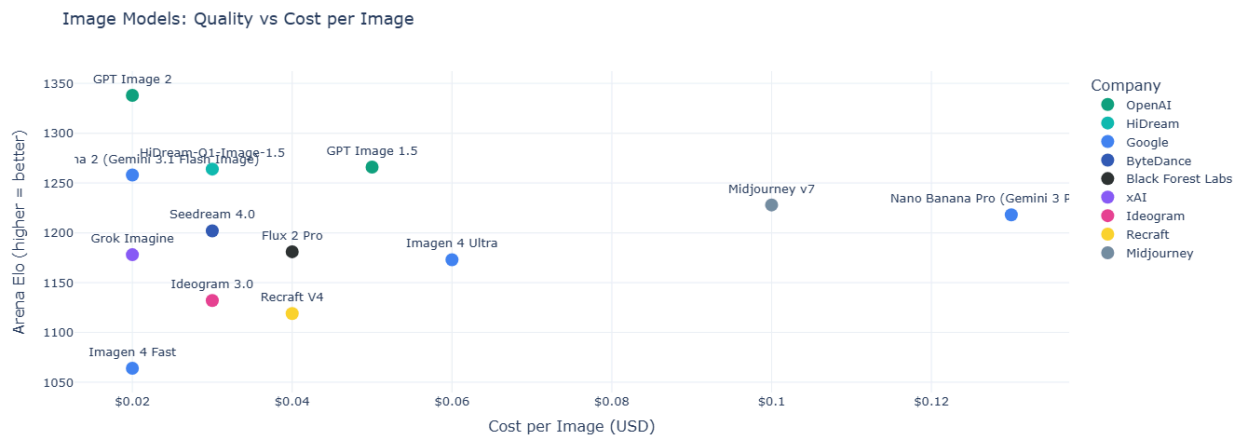
* The Arena Elo for Midjourney v7 and Ideogram 3.0 is an estimate. Midjourney doesn't have a public API and isn't part of the main Artificial Analysis arena, so the scores come from other ranking sources and aren't measured in the exact same way

Graph 11: Image Quality (Arena Elo)

The graph ranks the image models based on their Arena Elo, which is basically how often people preferred each model's images in a head to head voting. GPT Image 2 was a clear winner at 1338, much higher than every other model. After that there is a tight cluster of models around 1180 and 1265, so a lot of them are pretty close in quality. One thing I thought was cool is that HiDream, which is the only open weight model in the group, did really well at 1264.

Graph 12: Cost per Image

This graph shows much each model costs to generate one image. There is a big range here, from Nano Banana Pro at \$0.13 and Midjourney at \$0.10 all the way down to models costing \$0.02 per image. Something interesting is that the most expensive models aren't the highest quality ones, since Nano Banana Pro costs the most but did not win on quality.

Graph 13: Quality vs Cost per Image

This scatter plot compares image quality against cost, so the best value models are the ones that are high up and far to the left. GPT Image 2 is the obvious standout because it has the highest

ELo and is one of the cheapest models, which makes it the best of both worlds. On the other hand, Imagen 4 Fast is cheap but has the lowest quality, and Nano Banana Proof is expensive without being the best.

Section 4: Video Generation Models

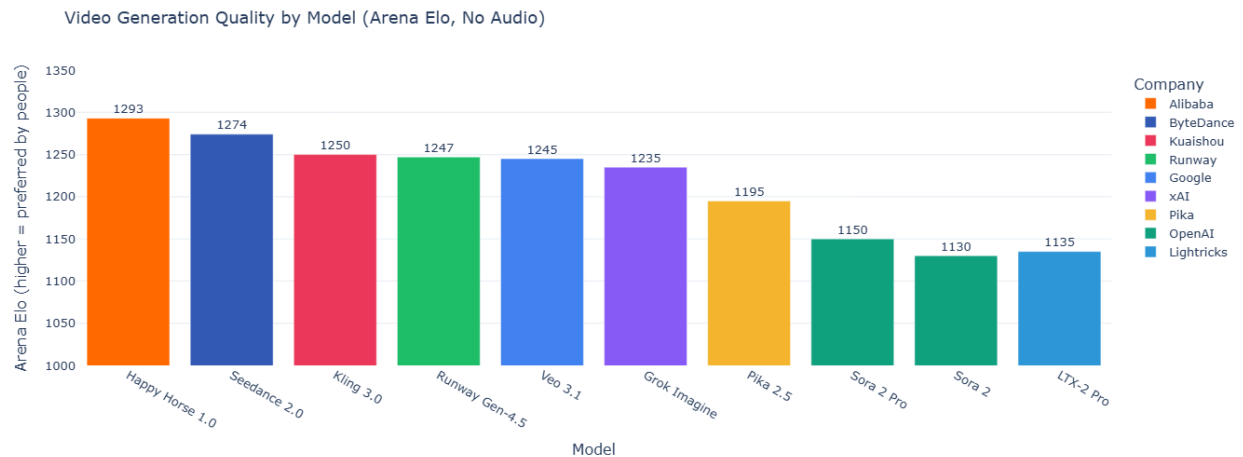
This section compares the top video generation models. Like image, video quality is judged using Arena Elo from people voting, but the scores move around a lot between sources so I focused on cost and the most reliable number too.

The Video Dataset

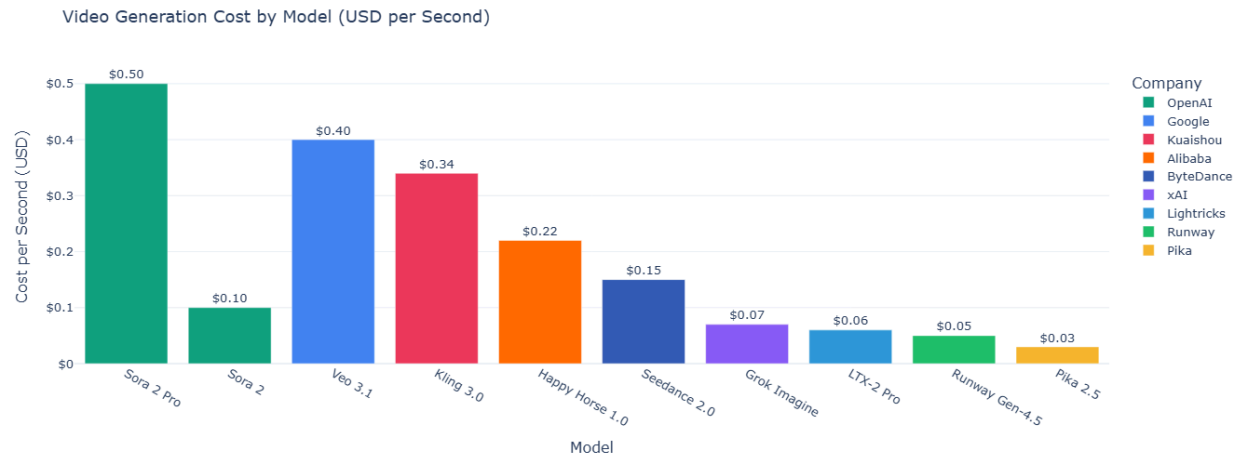
Model	Company	Arena Elo	Cost per Second	Max Clip Length	Max Resolution	Native Audio
Happy Horse 1.0	Alibaba	1293	\$0.22	10s	1080p	Yes
Seedance 2.0	ByteDance	1274	\$0.15	20s	1080p	Yes
Kling 3.0	Kuaishou	1250	\$0.34	15s	4K	Yes
Runway Gen-4.5	Runway	1247	\$0.05	10s	720p	No
Veo 3.1	Google	1245*	\$0.40	60s	4K	Yes
Grok Imagine	xAI	1235	\$0.07	15s	720p	Yes
Pika 2.5	Pika	1195*	\$0.03	5s	1080p	Limited
Sora 2 Pro	OpenAI	1150*	\$0.50	25s	1080p	Yes
LTX-2 Pro	Lightricks	1135	\$0.06	20s	4K	Yes
Sora 2	OpenAI	1130*	\$0.10	15s	1080p	Yes

*The no-audio Arena Elo for Veo 3.1, Pika 2.5, Sora 2 Pro, and Sora 2 is an estimate. These models showed up mostly on the with-audio leaderboards, so I estimated their no-audio scores based on their whey rank. Also, Sora 2 is being discontinued by OpenAI, so I included it as a baseline for comparison.

Graph 14: Video Quality (Arena Elo)



This graph ranks the video models by Arena Elo. Happy Horse 1.0 came out on top at 1293, with Seedance 2.0, Kling 3.0, Runway Gen-4.5, and Veo 3.1, and Grok Imagine all clustered close behind it. A lot of the newer models did really well, while Sora 2 ranked much lower than I expected, which might be part of why it is being discontinued.

Graph 15: Video Cost per Second

This graph shows how much each model costs per second of video, which is a big deal since video gets expensive fast. There is a huge range, from Sora 2 Pro at \$0.50 per second, and Veo 3.1 at \$0.40 down to Pika at only \$0.03. This shows how a one minute video could differ wildly in cost based on the model you use.

Graph 16: Quality vs Cost per Second

This scatter plot compares video quality against cost per one second, so the best value models are higher up and to the left. Runway Gen-4.5 and Grok Imagine stood out as great value

here, since both have high ELo but are some of the cheapest. On the other side, Veo 3.1, Kling 3.0, Sora 2 Pro are expensive, so you are paying a lot more for quality that is not always higher.

What I Found

For video models, the new ones like Happy Horse and Seedance led on quality, but the best value models were Runway Gen-4.5 and Grok Imagine since they are cheap and rank high. The cost difference between models was the biggest of any section, so price matters a lot here. I also learned that video quality is harder to measure than text and even image, since the scores change between sources, which is why I leaned on cost as more solid comparisons.

Conclusion

Overall this project allowed me to use Data Science and the python skills we learned in EECS to analyze a topic that I enjoy learning about. I learned how to create my own datasheet, how to organize data using Pandas, how to use three different graphing libraries (Plotly, Seaborn, and Matplotlib) for different types of charts, and how to actually understand the results once they were visualized.

The biggest thing I learned is that there is no single best AI model. At first I figured that all the models were basically the same except for the company that made them, but it turned out that different models are better than different things. For text, Claude Fable 5 was the smartest and best at coding, but Grok 4.3 was the best value. For images, GPT Image 2 was the best and still cheap. FOr video, the newer models like Happy Horse led on quality, while Runaway and Grok Imagine were the best deals overall. Almost every section came out to this sort of tradeoff between how good a model is, how fast it is, and how much it costs.

I also learned that not every type of AI is judged the same way. Text models have standardized benchmarks that test every model the same exact way, but image and video models are mostly judged by human preference through Arena Elo, which is less exact and changes more between sources. That is actually why a few of my images and video sources are estimates. This showed me how to estimate data while still being accurate by comparing different sources.